



Pet Tracker

Relatório de desenvolvimento do projeto concluído

AUTOR

Pedro Izkovitz

DATA

29 de maio de 2026

STATUS

MVP concluído e funcional

REPOSITÓRIO

github.com/pedroizk/pet-tracker

Disciplina · Projeto integrado · Relatório final

1. Visão geral

O **Pet Tracker** é uma aplicação web, concluída e funcional, que rastreia pets em tempo real e alerta o dono quando o animal sai de uma *zona segura*. O projeto partiu de um caso de estudo (o cachorro "Rei Julian", que foge de casa) e foi desenvolvido até um MVP completo que combina três mecanismos complementares de reencontro: **geofencing** preventivo, **link público de pet perdido** para mobilizar a vizinhança, e **reportes via QR code** que dispensam hardware GPS no animal.

Este relatório descreve o que foi efetivamente construído: a arquitetura final, as funcionalidades implementadas, a API HTTP, as principais decisões técnicas e o resultado obtido. A especificação inicial do projeto está no documento `caso-estudo.pdf`, no mesmo repositório.

2. Arquitetura

A aplicação roda como um único binário Go, sem instalação de dependências e sem serviços externos. O backend serve tanto a API quanto as páginas estáticas; o frontend é HTML/CSS/JS puro (vanilla) com mapa Leaflet sobre OpenStreetMap.

- **Backend (Go, ~1.380 linhas):** apenas a biblioteca padrão (`net/http`, `sync`, `encoding/json`) — o `go.mod` não tem nenhuma dependência externa. A concorrência é controlada por `sync.RWMutex` e `sync/atomic`.
- **Tempo real: Server-Sent Events (`/stream`).** Cada cliente recebe um canal próprio; o servidor faz *broadcast* de eventos (nova posição, pet criado/editado/ removido, alerta de zona) para todos os conectados. O frontend reconecta sozinho e mantém heartbeat de 15s.
- **Frontend (~2.700 linhas de HTML/CSS/JS):** 5 páginas — painel do dono, página pública de pet perdido, formulário de avistamento por QR, "celular como tracker" e mural público. Inclui dark mode com design tokens, layout responsivo, toasts e atalhos de teclado.
- **Persistência:** snapshot completo do estado em `store.json`, com auto-save disparado por flag *dirty* e gravação atômica (arquivo temporário + `rename`). O estado é recarregado no startup, então pets, zonas, históricos e tokens públicos sobrevivem a um restart.

3. Funcionalidades implementadas

Funcionalidade	Descrição	Status
Multi-pet	Cadastro de vários pets (cão, gato, coelho, pássaro, outro), com cor customizável e edição/exclusão.	✓ Concluído
Geofence + alertas	Zona segura circular desenhada no mapa; alerta (banner + beep + Web Notification) quando o pet sai dela.	✓ Concluído
Modo perdido + link público	Gera token de 128 bits e abre <code>/lost/{token}</code> com mapa em tempo real, colável no WhatsApp.	✓ Concluído
Reportes via QR code	Cada pet tem um QR; estranhos reportam a posição anonimamente em <code>/sight/{token}</code> , sem hardware.	✓ Concluído
Celular como tracker	<code>/sender</code> usa o GPS do navegador (<code>watchPosition</code>) com wake-lock e intervalo configurável.	✓ Concluído
Mural público	<code>/mural</code> lista os pets atualmente perdidos para a comunidade.	✓ Concluído
Estatísticas por pet	Distância total, tempo ativo e velocidade média, calculadas sobre o histórico.	✓ Concluído
Persistência em disco	Estado salvo em <code>store.json</code> com auto-save; sobrevive a restart.	✓ Concluído
UI / UX	Dark mode, sidebar responsivo, toasts, status do SSE, tutorial e atalhos de teclado.	✓ Concluído
Contas de dono (auth)	Autenticação por sessão — especificada e documentada, deixada como evolução futura.	○ Futuro

4. API HTTP

Principais endpoints expostos pelo backend (todos JSON, exceto as páginas HTML e o stream):

Rota	Função
GET/POST <code>/pets</code>	Lista e cria pets; PATCH/DELETE <code>/pets/{id}</code> edita e remove.
PUT/DELETE <code>/pets/{id}/zone</code>	Define e remove a zona segura.
POST/DELETE <code>/pets/{id}/lost</code>	Marca como perdido (gera token público) e como encontrado.
GET <code>/pets/{id}/stats</code>	Estatísticas de movimento do pet.
GET/POST <code>/position</code>	Lê a posição atual de todos os pets e ingere novas posições.
GET <code>/history</code>	Histórico de pontos de um pet (até 1000).
GET <code>/stream</code>	Server-Sent Events em tempo real.
GET <code>/lost/{token}/state</code>	Estado JSON do pet perdido, consumido pelo mapa público.
POST <code>/sight/{token}</code>	Recebe um avistamento anônimo via QR (posição + nota).
GET <code>/api/mural</code> · <code>/healthz</code>	Lista pública de perdidos e health check.

5. Decisões técnicas

- **Só biblioteca padrão do Go:** mantém o projeto sem build complexo — basta `go run .`.
Reforça os fundamentos de HTTP, JSON e concorrência sem esconder atrás de um framework.
- **SSE em vez de polling/WebSocket:** o fluxo é majoritariamente servidor → cliente (posições e alertas), então SSE entrega tempo real com reconexão nativa e muito menos código que WebSocket.
- **JSON em disco em vez de SQLite:** a especificação previa SQLite, mas o volume de dados é pequeno e cabe inteiro em memória. Um snapshot JSON com gravação atômica resolve a persistência sem dependência externa nem CGO, simplificando a entrega — uma simplificação consciente sobre o plano inicial.
- **Geofencing com fórmula de Haversine:** distância entre coordenadas calculada no servidor (raio da Terra = 6.371 km) para decidir se o pet está dentro ou fora da zona segura.

- **QR code como "dispositivo de rastreamento"**: o diferencial do projeto. Tokens públicos com escopo restrito permitem que terceiros reportem a posição sem hardware GPS no animal (~R\$ 5 contra R\$ 200+ de uma coleira GPS).

6. Resultado e conclusão

O MVP foi **concluído e está funcional**: compila sem erros, passa no `go vet` e roda com um único comando. A hipótese central do caso de estudo foi validada — *geofencing em tempo real funciona inteiramente no navegador, sem hardware proprietário* — e os dois diferenciais de baixíssimo custo (link público de pet perdido e reportes via QR code) estão implementados e operacionais.

Em relação ao plano inicial, todas as funcionalidades centrais foram entregues; a persistência foi simplificada de SQLite para JSON em disco, e a autenticação por contas de dono foi documentada e deixada como evolução futura, sem comprometer o uso standalone da aplicação.

Repositório: <https://github.com/pedroizk/pet-tracker>